Open in app ↗

Medium Search Write

Get unlimited access to the best of Medium for less than $1/week. Become a member

Engineering @ Housing/P...

Auto-Scaling based on Prometheus Custom Metrics

Subhi Singh Follow 3 min read · Dec 20, 2022

63 1

In Kubernetes, whenever we have to scale our application, we have tendency to do it based on resource usage i.e. CPU and Memory.

But, what if scaling based on CPU or Memory will no longer be criteria for our application ?

Then what are the options available in our hand ? -

1. Let's not scale it or auto-scale on cpu and/or memory 🙁

2. Vertical scale 🙂

3. Look out if we can stop the application 😣

Don't worry, you don't have to accept any of the three options as Prometheus already thought of this kinda scenario and introduced Prometheus Adapter to ease out our life.

Now two questions can come to your mind -

1. What is this prometheus adapter ?

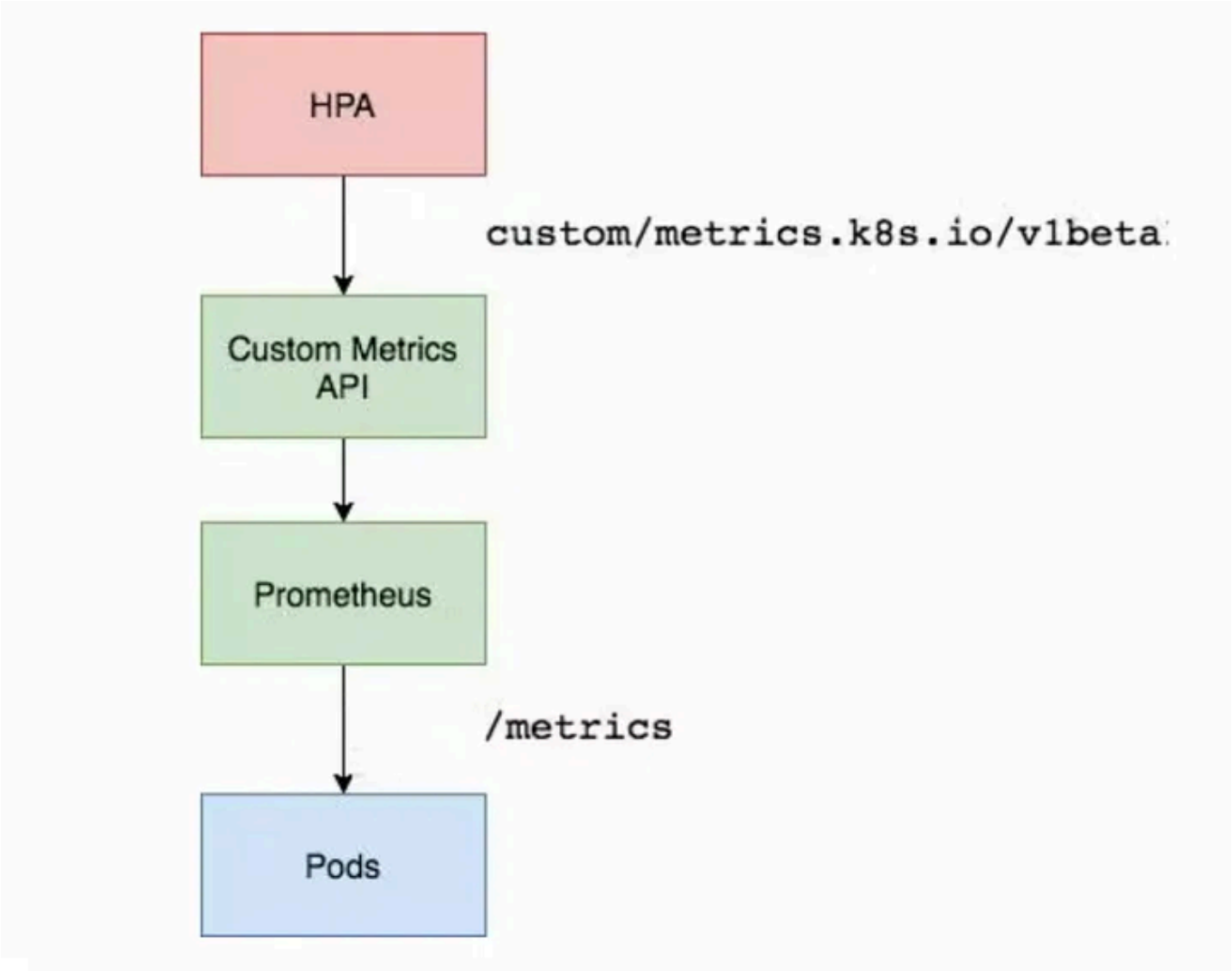
2. How to use it to scale our application ?

Let's dive in.

We all know that Prometheus is the standard tool for monitoring deployed workloads and the Kubernetes cluster itself. Prometheus adapter helps us to leverage the metrics collected by Prometheus and use them to make scaling decisions.

These metrics are exposed by an API service and can be readily used by our **Horizontal Pod Autoscaling object**.

We will be using Prometheus adapter to pull custom metrics from our Prometheus installation and then let the Horizontal Pod Autoscaler (HPA) use it to scale the pods up or down.



Architecture Overview

If you want to know more about how prometheus adapter works, please check this out -

setup prometheus-adapter — https://github.com/kubernetes-sigs/prometheus-adapter/blob/master/docs/walkthrough.md

The adapter determines which metrics to expose, and how to expose them, through a set of “discovery” rules. Each rule is executed independently (so make sure that your rules are mutually exclusive), and specifies each of the steps the adapter needs to take to expose a metric in the API.

Each rule can be broken down into roughly four parts:

> **Discovery**, *which specifies how the adapter should find all Prometheus metrics for this rule.*
>
> **Association**, *which specifies how the adapter should determine which Kubernetes resources a particular metric is associated with.*
>
> **Naming**, *which specifies how the adapter should expose the metric in the custom metrics API.*
>
> **Querying**, *which specifies how a request for a particular metric on one or more Kubernetes objects should be turned into a query to Prometheus.*

More details on these are here — https://github.com/kubernetes-sigs/prometheus-adapter/blob/master/docs/config.md#metrics-discovery-and-presentation-configuration

Let's see one example here -

We know, for some Java applications memory usage based scaling doesn't work always. So, we thought to scale our applications based on the **average JVM usage**. In order to do that -

***** Deploy prometheus adapter in your kubernetes cluster.*****

Refer this https://github.com/kubernetes-sigs/prometheus-adapter

1. First we have to make sure the metrics are present in Prometheus. These are the metrics we will be looking for -

```
jvm_memory_used_bytes and jvm_memory_max_bytes
```

2. Next we have to create a rule in Prometheus Adapter

```
    - seriesQuery: '{__name__=~"jvm_memory_bytes_(used|max)",area="heap"}'
      seriesFilters:
      - is: ^jvm_memory_bytes_(used|max)$
      resources:
        overrides:
          namespace:
            resource: namespace
          service:
            resource : service
          pod:
            resource : pod
      name:
        matches: ^jvm_memory_bytes_(used|max)$
        as: "jvm_used_percent_housing"
      metricsQuery: ((sum((jvm_memory_used_bytes{area="heap", <<.LabelMatchers>>
```

3. Once you deploy this using Prometheus Adapter, it should be available in your API Service. You can check via this command -

```
kubectl get - raw "/apis/custom.metrics.k8s.io/v1beta1" | jq .
```

4. Now we can use **jvm_used_percent_housing** rule to scale our application. In order to do that we have to write our HPA template -

```yaml
apiVersion: autoscaling/v2beta2
kind: HorizontalPodAutoscaler
metadata:
  name: ApplicationName
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ApplicationName
  minReplicas: 1
  maxReplicas: 2
  metrics:
  - type: Pods
    pods:
      name: jvm_used_percent_housing
      target:
        type: AverageValue
        averageValue: 80m
```

Note: Here averageValue: 80m means whenever average JVM usage of application is > 80%, it will start scaling.

Conclusion

THERE IT IS !!🎉. Now you are no longer bound to use only CPU and Memory as scaling criteria. You can add your own metrics and based on that you can scale your application by using Prometheus Adapter.

Kubernetes | Cluster Autoscaler | Prometheus

Published in Engineering @ Housing/Proptiger/Makaan

1.4K followers · Last published Jun 11, 2024

Engineering and technology articles from developers

Follow

Written by Subhi Singh

3 followers · 3 following

https://www.linkedin.com/in/subhi-singh-5612ba189

Follow

Responses (1)

Anand Ray

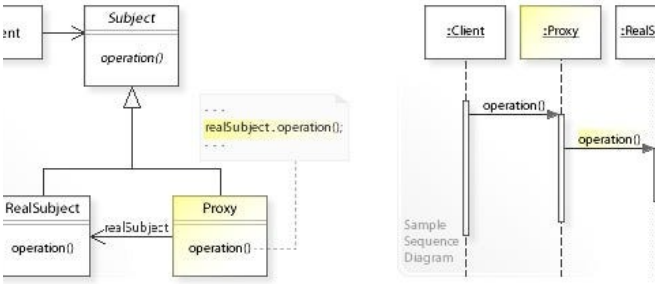
What are your thoughts?

Lucas Fernando Cardoso Nunes
Mar 27

The correct regex is `^jvm_memory_(used|max)_bytes$`

Reply

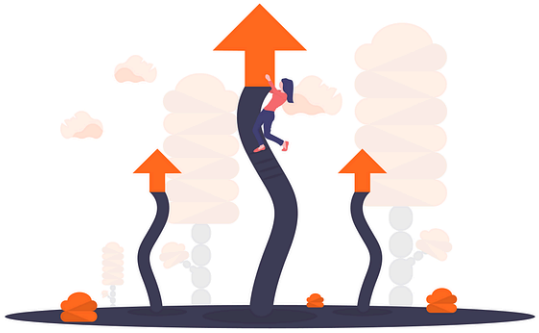
More from Subhi Singh and Engineering @ Housing/Proptiger/Makaan



In Engineering @ Housing/Propti... by somesh sh...

Proxy Pattern and Spring @ Transactional

This article will give a brief introduction about the Proxy Pattern and it's application in Sprin...



In Engineering @ Housing/Proptig... by Priyam G...

Migrating from Spring To Spring Boot

You heard it right, we have recently upgraded our backend Spring application to...

Dec 21, 2022 52 2

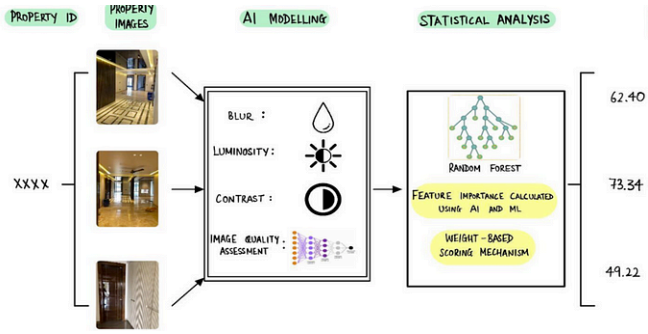
In Engineering @ Housing/Proptig... by Rahul Ma...

React Native Measuring and Improvising App Performance

Measuring React Native performance can be done by using tools such as the Flipper and...

Apr 26, 2023 117

Dec 7, 2022 19



In Engineering @ Housing/Propti... by Prateek Ch...

Image Scoring: Allocating Percentage Score to Images for...

This article illustrates a way to quantify the image quality based on certain image qualit...

Apr 19, 2022 189

See all from Subhi Singh

See all from Engineering @ Housing/Proptiger/Makaan

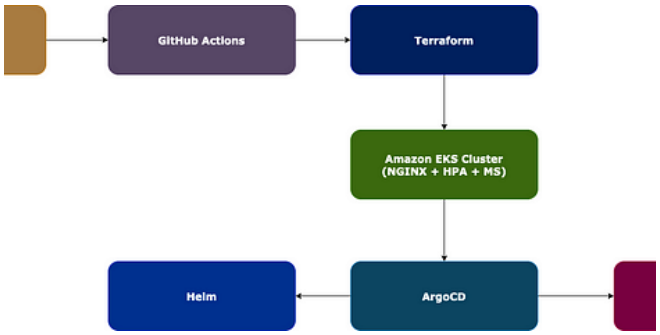
Recommended from Medium

NetOpsChic

Deploying Kube-state-metric, Prometheus, and Grafana for...

Deploying applications on a Kubernetes cluster involves managing numerous...

Jan 11 3



Ram Vadranam

Mastering Horizontal Pod Autoscaling (HPA) in Amazon EK...

In the ever-evolving cloud-native landscape, ensuring application scalability while...

Nov 29, 2024 35

In DevPulse by Rsprasangi

The Ultimate Monitoring Stack You Should Be Using in 2025!

From sleepless nights troubleshooting production fires to building observability-firs...

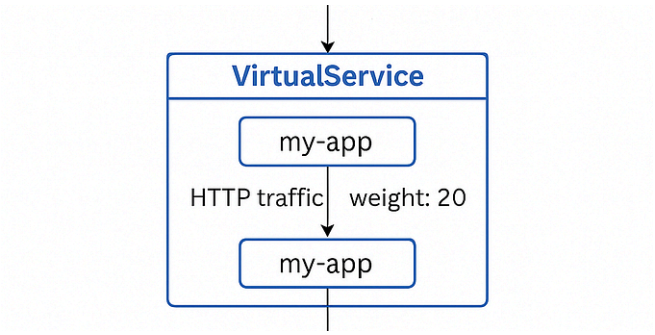
May 6 2

Nirbhay Singh

The 2025 DevOps Revolution: How AI is Transforming Infrastructure...

Introduction:

Dec 16, 2024



DevOpsDynamo

Understanding Istio Traffic Routing: Gateways,...

When working with Kubernetes, controlling traffic between services can get complicate...

Apr 18 177 1

Matías Salinas

Understanding Prometheus Metrics

Prometheus is an open-source monitoring system designed to collect, store, and query...

May 1 17

See more recommendations